

BIKE UNIT (RECIVER)

```
#include <WiFi.h>
```

```
#include <esp_now.h>
```

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
// — WiFi (same hotspot as helmet!) —————
```

```
const char* ssid  = "YOUR_HOTSPOT_NAME";
```

```
const char* password = "YOUR_HOTSPOT_PASSWORD";
```

```
// — Pins —————
```

```
#define BUZZER    26
```

```
#define WARNING_LED  27
```

```
#define RELAY_PIN   33
```

```
#define IGNITION_LED 25
```

```
// — LCD —————
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

```
// — Data packet —————
```

```
typedef struct {
```

```
    bool helmetWorn;
```

```
    bool alcoholDetected;
```

```
    int  alcoholADC;
```

```
    bool dataValid;
```

```
} DataPacket;
```

```
DataPacket received;
```

```
// — Timing —————
```

```

unsigned long lastPacketTime = 0;

unsigned long helmetRemovedAt = 0;

unsigned long startupTime = 0;


bool helmetWasRemoved = false;

bool bikeForceOff = false;

bool systemReady = false;


#define TIMEOUT_MS 8000

#define HELMET_GRACE_MS 15000

#define STARTUP_DELAY 5000


// — ESP-NOW receive callback (core 3.x) —————
void onReceive(const esp_now_recv_info_t *recv_info, const uint8_t *incomingData, int len) {
    memcpy(&received, incomingData, sizeof(received));

    lastPacketTime = millis();

    systemReady = true;

    Serial.println(" 📦 Packet received!");
}


// — Helpers —————
void allOff() {
    digitalWrite(RELAY_PIN, LOW);
    digitalWrite(IGNITION_LED, LOW);
    digitalWrite(WARNING_LED, LOW);
    digitalWrite(BUZZER, LOW);
}


void ignitionON() {
    digitalWrite(RELAY_PIN, LOW);
    digitalWrite(IGNITION_LED, HIGH);

```

```
digitalWrite(WARNING_LED, LOW);  
digitalWrite(BUZZER, LOW);  
}
```

```
void ignitionOFF() {  
    digitalWrite(RELAY_PIN, HIGH);  
    digitalWrite(IGNITION_LED, LOW);  
}
```

```
void warningON() {  
    digitalWrite(WARNING_LED, HIGH);  
    digitalWrite(BUZZER, HIGH);  
}
```

```
void warningOFF() {  
    digitalWrite(WARNING_LED, LOW);  
    digitalWrite(BUZZER, LOW);  
}
```

```
void lcdPrint(const char* line1, const char* line2) {  
    lcd.clear();  
    lcd.setCursor(0, 0); lcd.print(line1);  
    lcd.setCursor(0, 1); lcd.print(line2);  
}
```

```
void setup() {  
    Serial.begin(115200);  
    delay(1000);
```

```
    pinMode(BUZZER, OUTPUT);  
    pinMode(WARNING_LED, OUTPUT);
```

```

pinMode(RELAY_PIN, OUTPUT);
pinMode(IGNITION_LED, OUTPUT);
allOff();

Wire.begin(21, 22);
lcd.init();
lcd.backlight();
lcdPrint("Smart Helmet", "System v1.0");
delay(2000);
lcdPrint("Waiting for", "Helmet Unit...");

// — WiFi (same hotspot = same channel!) —————
WiFi.mode(WIFI_AP_STA);
WiFi.begin(ssid, password);
Serial.print("Connecting to WiFi");
int tries = 0;
while (WiFi.status() != WL_CONNECTED && tries < 20) {
    delay(500); Serial.print("."); tries++;
}
if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\nWiFi Connected ");
    Serial.print("Channel: "); Serial.println(WiFi.channel());
} else {
    Serial.println("\nWiFi FAILED ");
}

Serial.print("Bike MAC: "); Serial.println(WiFi.macAddress());

// — ESP-NOW —————
if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW Init FAILED!");
}

```

```

    lcdPrint("ESP-NOW FAIL", "Restart Bike");

    return;
}

esp_now_register_recv_cb(onReceive);

startupTime = millis();
lastPacketTime = millis();
Serial.println("Bike ready! 🚲");
}

void loop() {
    bool helmet = received.helmetWorn;
    bool alcohol = received.alcoholDetected;
    int adc = received.alcoholADC;
    bool valid = received.dataValid;

    bool connectionLost = systemReady && (millis() - lastPacketTime > TIMEOUT_MS);

    if (!systemReady && (millis() - startupTime < STARTUP_DELAY)) {
        allOff();
        delay(500);
        return;
    }

    if (!connectionLost && valid && helmet && !alcohol) {
        helmetWasRemoved = false;
        helmetRemovedAt = 0;
        bikeForceOff = false;
    }
}

```

```
if (!connectionLost && valid && !helmet && !alcohol) {  
    if (!helmetWasRemoved) {  
        helmetWasRemoved = true;  
        helmetRemovedAt = millis();  
        Serial.println("Helmet removed - 15s countdown!");  
    }  
    if (millis() - helmetRemovedAt >= HELMET_GRACE_MS) {  
        bikeForceOff = true;  
        Serial.println("15s passed - ignition cut!");  
    }  
}
```

// — CONDITION 1: Disconnected —————

```
if (connectionLost || !valid) {  
    ignitionOFF();  
    warningON();  
    lcdPrint("HELMET UNIT", "DISCONNECTED!");  
    Serial.println("STATE: Disconnected");  
}
```

// — CONDITION 2: Alcohol —————

```
else if (alcohol) {  
    ignitionOFF();  
    warningON();  
    lcd.clear();  
    lcd.setCursor(0, 0); lcd.print("ALCOHOL FOUND!");  
    lcd.setCursor(0, 1); lcd.print("ADC:"); lcd.print(adc); lcd.print(" BLOCKED");  
    Serial.print("STATE: Alcohol | ADC="); Serial.println(adc);  
}
```

// — CONDITION 3: Force cut —————

```
else if (bikeForceOff) {  
    ignitionOFF();  
    warningON();  
    lcdPrint("HELMET OFF 15s", "IGNITION CUT!");  
    Serial.println("STATE: Force cut");  
}
```

```
// — CONDITION 4: Grace countdown —————
```

```
else if (helmetWasRemoved && !bikeForceOff) {  
    unsigned long remaining = (HELMET_GRACE_MS - (millis() - helmetRemovedAt)) / 1000;  
    ignitionON();  
    warningON();  
    lcd.clear();  
    lcd.setCursor(0, 0); lcd.print("WEAR HELMET!");  
    lcd.setCursor(0, 1); lcd.print("STOP IN "); lcd.print(remaining); lcd.print("s ");  
    Serial.print("STATE: Grace | "); Serial.print(remaining); Serial.println("s left");  
}
```

```
// — CONDITION 5: Helmet not worn —————
```

```
else if (!helmet) {  
    ignitionOFF();  
    warningOFF();  
    digitalWrite(WARNING_LED, HIGH);  
    lcdPrint("WEAR HELMET!", "BIKE STOPPED");  
    Serial.println("STATE: Helmet not worn");  
}
```

```
// — CONDITION 6: All clear —————
```

```
else {  
    ignitionON();  
    warningOFF();  
}
```

```
    lcdPrint("BIKE STARTED", "RIDE SAFE! :)");  
    Serial.println("STATE: All clear ✅");  
}  
  
delay(500);  
}
```